Project Report - Place Mint (Placement Portal Application)

Author

- **Full Name:** Hemang
- **Roll Number:** 23f3001304
- **Student Email:** 23f3001304@ds.study.iitm.ac.in
- **About Me:** BS Data Science & Applications student at IIT Madras, interested in full-stack web development.

Description

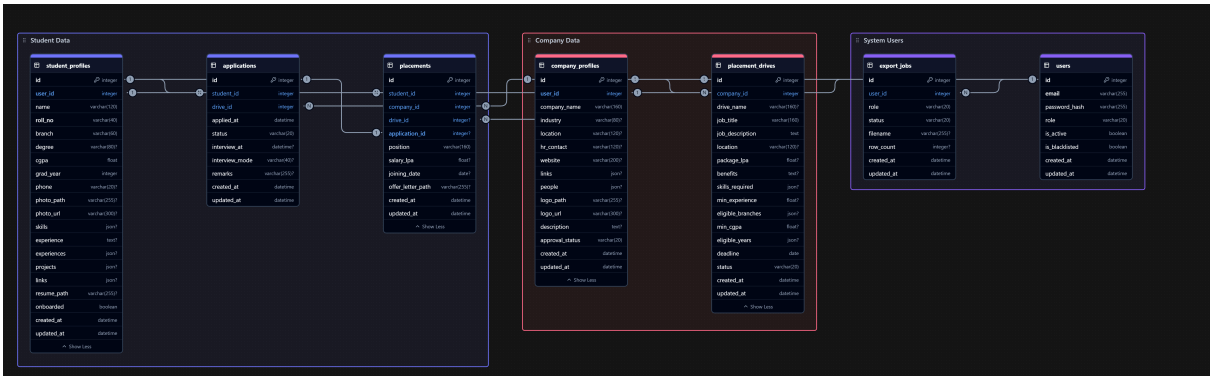
PlaceMint is a role-based campus recruitment portal replacing spreadsheet-based workflows with unified dashboards for Admins, Companies, and Students, featuring eligibility checks, analytics, and Celery jobs.

Technologies Used

- **Flask & Flask-SQLAlchemy:** Core REST API and SQLite ORM integration.
- **Flask-JWT-Extended:** Token-based authentication and role-based access control (RBAC).
- **Flask-Caching (with Redis):** Cache storage to optimize dashboard read statistics.
- **SQLite & Redis:** SQLite for relational storage; Redis as cache store and Celery message broker.
- **Celery & Celery Beat:** Background task execution for reminders, reports, and CSV exports.
- **Vue 3, Vite, & Bootstrap 5:** SPA frontend with responsive templates and layouts.
- **Chart.js & ReportLab:** Visual charts on dashboards and PDF report exports.

DB Schema Design

The database structure is programmatically managed by SQLAlchemy. The diagram below illustrates the entity-relationship design:



Design Rationale

- **Unified Auth & Decoupled Profiles:** Unified `users` credentials table with 1:1 profiles avoids data pollution.
- **First-Class Placement History:** Separated `placements` allows logging manual/off-campus offers.
- **Strict Double-Application Guard:** Unique `(student_id, drive_id)` constraint prevents concurrent duplicate applications.

API Design

Backend routes are grouped under role-based Blueprints. RBAC is verified per-route via decorators.

API Module	URL Prefix	Target Roles	Description & Core Handlers
Authentication	`/api/auth`	Public, Authenticated	User registration, login, and rate-limited auth sessions.
Admin Module	`/api/admin`	Admin	Dashboard stats, user moderation (blacklist/activate), approvals.
Company Module	`/api/company`	Company	Overview stats, profile updates, drive posting, and applicant review.
Student Module	`/api/student`	Student	Profile management (resume uploads), drive browsing, applying.
Shareable Directories	`/api/directory`	Admin, Company, Student	Shareable read-only profiles for companies and students.
Document Delivery	`/api/documents`	Public, Authenticated	Secure file rendering for logos, resumes, and offer letters.
Asynchronous Exports	`/api/exports`	Company, Student	Triggering, polling, and downloading Celery-generated CSV exports.

Architecture and Features

Project Organization

The decoupled SPA and REST API follow this folder structure:

```
PlaceMint/
├── backend/app/
│   ├── api/       # REST Blueprints (auth, admin, company, student, directory, etc.)
│   ├── auth/      # Password hashing, JWT credentials, and RBAC decorator
│   ├── caching/   # Redis cache keys and read-through caching fallback helpers
│   ├── models/    # SQLAlchemy database schema definition tables
│   ├── services/  # Business logic (eligibility matching, state transition)
│   └── tasks/     # Celery background tasks (reminders, reports, CSV exports)
└── frontend/src/
    ├── api/        # Client fetch network requests mapping to REST endpoints
    ├── components/ # Reusable frontend layout widgets, charts, and overlays
    ├── views/      # Dashboard pages for Admins, Companies, and Students
    ├── store/      # Global reactive state (auth, toast notification stores)
    └── router/     # Navigation routing, access guards, and role verification
```

Implemented Features

- **Core:** Role-based dashboards, automated eligibility checks, application tracking, admin moderation/blacklist.
- **Additional:** Celery daily reminders & monthly reports, ATS-style resume screener, asynchronous CSV export, PWA support, and Redis caching backend.

Video

- **Video Demonstration:** Placement Portal Walkthrough Video

AI/LLM Declaration

1. AI has been used to make this report.
2. AI has been used to refactor frontend UI and add 3D buttons.
3. AI has been used to enhance backend logic to implement repetitive tasks such as blueprint creation.
4. AI has been used to write tests.